

Module 6: Building AI APIs and Backends with FastAPI

Demo 1: Building Your First FastAPI LLM Endpoint

Objective

By the end of this hands-on, you will be able to:

- Create a FastAPI application for serving LLM-powered API endpoints
- Configure environment variables securely for AI application development
- Define Pydantic request and response models for structured API design
- Build an LLM-powered /generate endpoint using the Groq API
- Return structured JSON responses with generated output, model details, and status
- Run and test the FastAPI application using Uvicorn and Swagger UI
- Validate API requests automatically using Pydantic schemas
- Consume the API programmatically using Python requests
- Understand the complete request-response lifecycle of an AI backend service

Introduction

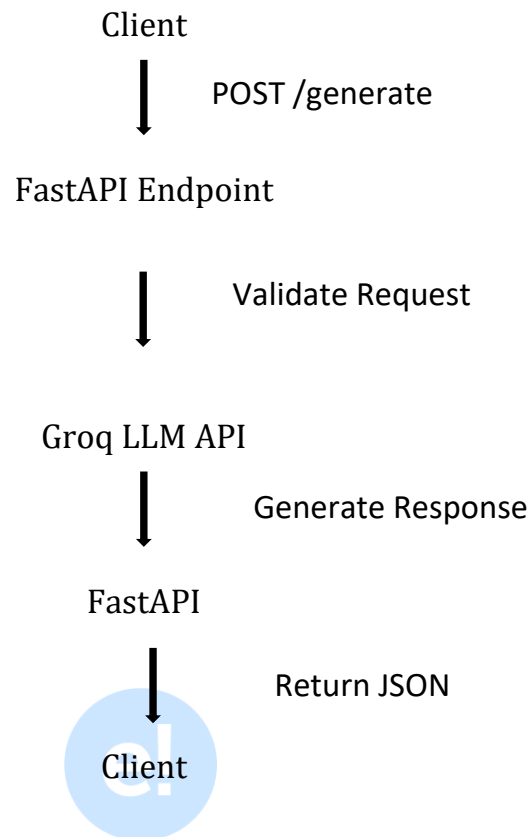
Most AI applications communicate through APIs. Whether the front-end is a chatbot, web application, mobile app, or automation workflow, the AI model is typically accessed through an API endpoint.

FastAPI is a modern Python framework for building APIs quickly and efficiently. It provides automatic validation, interactive documentation, and excellent performance.

In this hands-on, you will build a FastAPI application that accepts a prompt, sends it to the Groq LLM, and returns the generated response as JSON.

Application Architecture

Before writing code, understand the flow of the application.

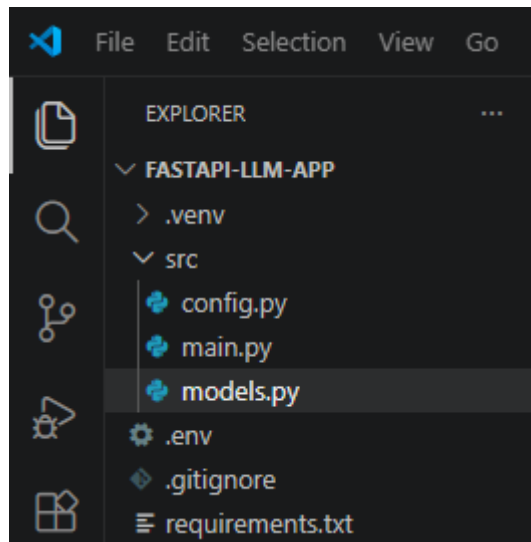


Request Flow

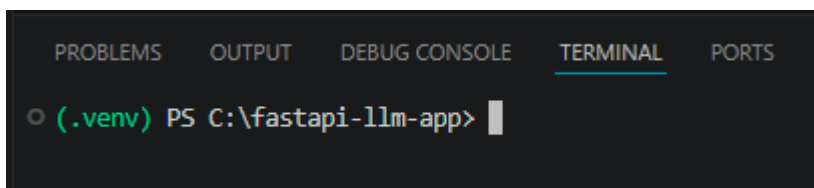
1. A client sends a prompt to the API.
2. FastAPI receives and validates the request.
3. The endpoint sends the prompt to Groq.
4. Groq generates a response.
5. FastAPI returns the response as JSON.

Project Setup

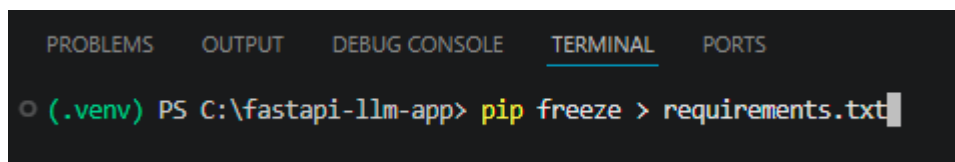
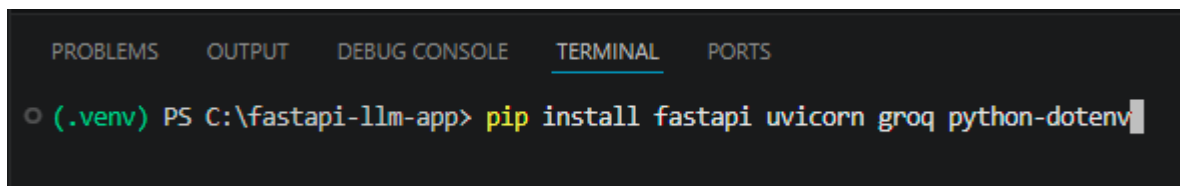
1. Create a new project folder and the required files for the FastAPI application.



2. Create and activate the virtual environment



3. Install the required packages and Save the dependencies.



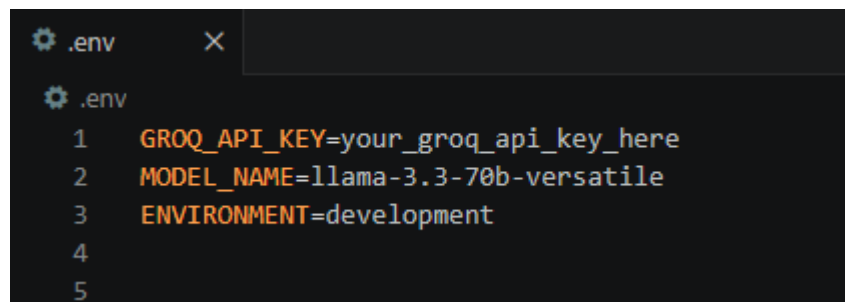
Package Overview

Package	Purpose
fastapi	Builds the backend API
uvicorn	Runs the FastAPI server
groq	Groq SDK
python-dotenv	Loads environment variables

4. Configure Environment Variables

- Open the .env file.
- Add:

```
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
MODEL_NAME=llama-3.3-70b-versatile
ENVIRONMENT=development
```

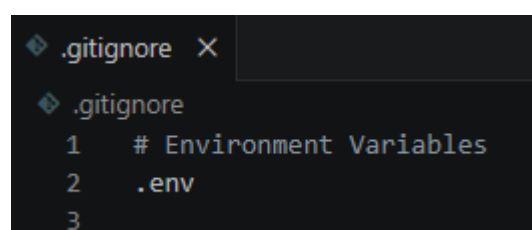


```
.env
1 GROQ_API_KEY=your_groq_api_key_here
2 MODEL_NAME=llama-3.3-70b-versatile
3 ENVIRONMENT=development
4
5
```

- Replace the API key with your own key.
- Save the file.

5. Protect Sensitive Files

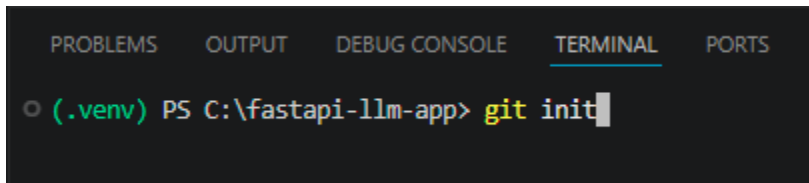
- Open .gitignore
- If the file is empty or does not already contain the following entry, add:



```
.gitignore
1 # Environment Variables
2 .env
3
```

6. Verify Git Protection

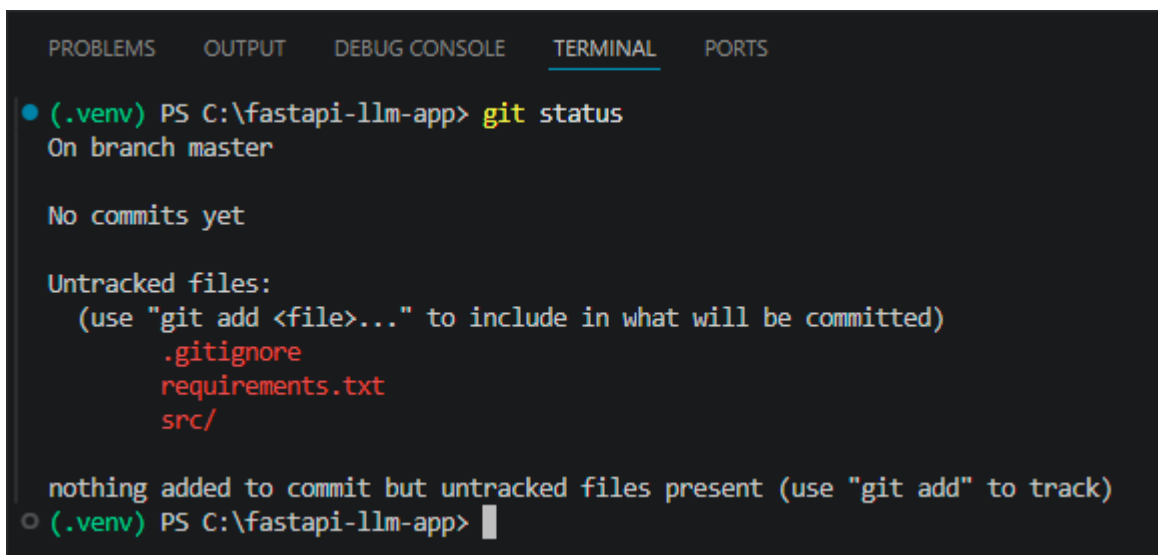
- Initialize Git



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(.venv) PS C:\fastapi-llm-app> git init
  
```

- Run git status command



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(.venv) PS C:\fastapi-llm-app> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        requirements.txt
        src/

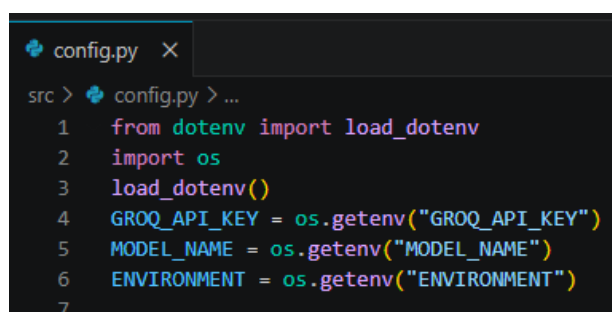
nothing added to commit but untracked files present (use "git add" to track)
(.venv) PS C:\fastapi-llm-app>
  
```

What to Observe

The .env file does not appear in the list of untracked files because it is excluded by .gitignore. This confirms that sensitive configuration values and API keys will not be tracked by Git.

7. Create the Configuration Module

- **src/config.py**
- Add the following code **config.py**



```

config.py  X
src > config.py > ...
1  from dotenv import load_dotenv
2  import os
3  load_dotenv()
4  GROQ_API_KEY = os.getenv("GROQ_API_KEY")
5  MODEL_NAME = os.getenv("MODEL_NAME")
6  ENVIRONMENT = os.getenv("ENVIRONMENT")
7
  
```

Understanding Pydantic Models

Pydantic models define the structure of incoming requests.

Expected request:

```
{
  "prompt": "Explain Agentic AI"
}
```

FastAPI automatically validates the request body against this schema.

Why Use Pydantic?

Without validation:

```
{
  "wrong_field": "hello"
}
```



could reach the application.

With Pydantic, invalid requests are automatically rejected.

Building the API Components

Step 1 – Create Pydantic Request and Response Models

- Open **src/models.py**
- Add the following Code:

```
models.py U X
src > models.py > ...
1  from pydantic import BaseModel
2
3  class PromptRequest(BaseModel):
4      prompt: str
5
6
7  class PromptResponse(BaseModel):
8      response: str
9      model: str
10     status: str
```

Code explanation

```
from pydantic import BaseModel
```

Imports the **BaseModel** class from **Pydantic**. All request and response schemas in FastAPI are typically created by inheriting from **BaseModel**.

```
class PromptRequest(BaseModel):
```

Defines a Pydantic model named **PromptRequest**. This model represents the expected structure of incoming API requests.

```
    prompt: str
```

Declares a field named **prompt** of type string. Every request sent to the API must include this field.

Example Request

```
{
  "prompt": "Explain Agentic AI"
}
```



```
class PromptResponse(BaseModel):
```

Defines a Pydantic model named **PromptResponse**. This model represents the structure of the response returned by the API.

```
    response: str
```

Stores the text generated by the LLM.

```
    model: str
```

Returns the model name used to generate the response.

```
status: str
```

Indicates whether the request was processed successfully.

Expected Response Format

```
{  
  
  "response": "Agentic AI refers to...",  
  
  "model": "llama-3.3-70b-versatile",  
  
  "status": "success"  
}
```

What This Model Does

The PromptRequest model validates the data coming into the API, while the PromptResponse model defines the structure of data sent back to the client.

Together, they make the API more predictable, easier to test, and closer to production-ready API design. FastAPI uses these models to validate inputs, document schemas, and return structured responses.

Step 2- Create the FastAPI Application

- Open **src/main.py**
- Add the following Code:

```

1  from fastapi import FastAPI, HTTPException
2  from groq import Groq
3
4  from config import GROQ_API_KEY, MODEL_NAME
5  from models import PromptRequest, PromptResponse
6
7
8  app = FastAPI(
9      title="FastAPI LLM Application",
10     description="A simple FastAPI application powered by Groq",
11     version="1.0.0"
12 )
13
14 client = Groq(api_key=GROQ_API_KEY)
15
16
17 @app.get("/")
18 def home():
19     return {
20         "message": "FastAPI LLM Application is running."
21     }
22
23
24 @app.post(
25     "/generate",
26     response_model=PromptResponse
27 )
28

```

```

29 def generate_response(request: PromptRequest):
30     try:
31         response = client.chat.completions.create(
32             model=MODEL_NAME,
33             messages=[
34                 {
35                     "role": "user",
36                     "content": request.prompt
37                 }
38             ],
39             temperature=0.2,
40             max_tokens=300
41         )
42
43         generated_text = (
44             response.choices[0]
45             .message
46             .content
47         )
48
49         return {
50             "response": generated_text,
51             "model": MODEL_NAME,
52             "status": "success"
53         }
54
55     except Exception as error:
56         raise HTTPException(
57             status_code=500,
58             detail=str(error)
59         )

```

Code explanation

```
from fastapi import FastAPI, HTTPException
```

Imports the core FastAPI class and HTTPException. FastAPI is used to create the API application, while HTTPException is used to return clear error responses if something goes wrong.

```
from models import PromptRequest, PromptResponse
```

Imports both Pydantic models. PromptRequest validates the incoming request body, while PromptResponse defines the structure of the API response.

```
app = FastAPI()
```

Creates the FastAPI application object. All API routes are attached to this object.

```
client = Groq(api_key=GROQ_API_KEY) FastAPI()
```

Creates a connection to the Groq API.

```
@app.get("/")
```

Creates a simple endpoint that confirms the application is running.

Request:

GET /

Response:

```
{  
  "message": "FastAPI LLM Application is running."  
}
```

```
@app.post("/generate", response_model=PromptResponse)
```

Creates an endpoint that accepts prompts and generates AI responses. The `response_model=PromptResponse` part tells FastAPI that the response should follow the structure defined in the `PromptResponse` model. This makes the API output more predictable and improves the automatically generated Swagger documentation.

```
def generate_response(request: PromptRequest):
```

Defines the API function that handles requests sent to the `/generate` endpoint. The `request` parameter uses the `PromptRequest` Pydantic model, allowing FastAPI to validate incoming data automatically.

```
try:
    response = client.chat.completions.create(
        model=MODEL_NAME,
        messages=[
            {
                "role": "user",
                "content": request.prompt
            }
        ],
        temperature=0.2,
        max_tokens=300
    )
```

- **try:** starts a try-except block that handles unexpected errors during the API request.
- **response = client.chat.completions.create()** creates a chat completion request and sends it to the Groq API.

- **model=MODEL_NAME** specifies which language model should process the request.
- **messages** contains the conversation sent to the model. In this example, a single user message is provided using the prompt received from the API request.
- **request.prompt** retrieves the prompt submitted by the client through the FastAPI endpoint.
- **temperature=0.2** controls response randomness. Lower values produce more focused and consistent responses.
- **max_tokens=300** limits the maximum length of the generated response.
- The returned response object contains the text generated by the model and additional metadata about the request.

```
generated_text = (
    response.choices[0]
    .message
    .content
)
return {
    "response": generated_text
    "model": MODEL_NAME,
    "status": "success"
}
```

- **generated_text = (** creates a variable to store the text generated by the language model.
- **response.choices[0]** accesses the first response returned by the model.
- **.message** retrieves the message object from the response.
- **.content** extracts the actual generated text from the LLM.
- **generated_text** now contains the final response produced by the model.

- **return {** creates a Python dictionary that will be returned to the client.
- **"response":** the generated LLM output
- **model :** the model used to generate the response
- **status :** whether the request was successful
- FastAPI automatically converts the returned Python dictionary into a JSON response.

```
except Exception as error:
    raise HTTPException(
        status_code=500,
        detail=str(error)
    )
```

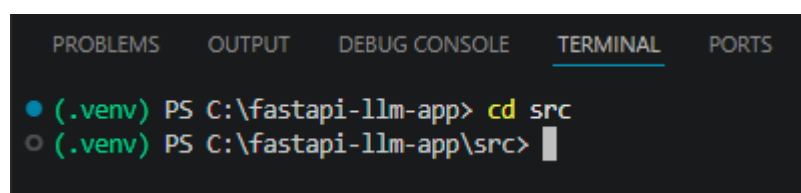
Captures unexpected failures and returns an API-friendly error response.

Step 3 - Run the Application

Before starting the FastAPI server, navigate to the src directory where the application files are located.

Run:

```
cd src
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) PS C:\fastapi-llm-app> cd src
(.venv) PS C:\fastapi-llm-app\src> |
```

Start the FastAPI server.

```
uvicorn main:app --reload
```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(.venv) PS C:\fastapi-llm-app\src> uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\fastapi-llm-app\\src']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [19812] using StatReload
INFO:     Started server process [14196]
INFO:     Waiting for application startup.
INFO:     Application startup complete.

```

Note

In this project, **main.py**, **config.py**, and **models.py** are located inside the **src** folder. Therefore, the application is started from within the **src** directory.

Alternatively, if the project is structured as a Python package, the application can be started from the project root using:

```
uvicorn src.main:app --reload
```

In that case, imports inside main.py should be updated to:

```

from src.config import GROQ_API_KEY, MODEL_NAME
from src.models import PromptRequest, PromptResponse

```

Understanding the Uvicorn Command

```
uvicorn src.main:app --reload
```

Part	Meaning
uvicorn	Application server
main	Python file containing the FastAPI application (main.py)
app	FastAPI application object
--reload	Automatically reloads when code changes

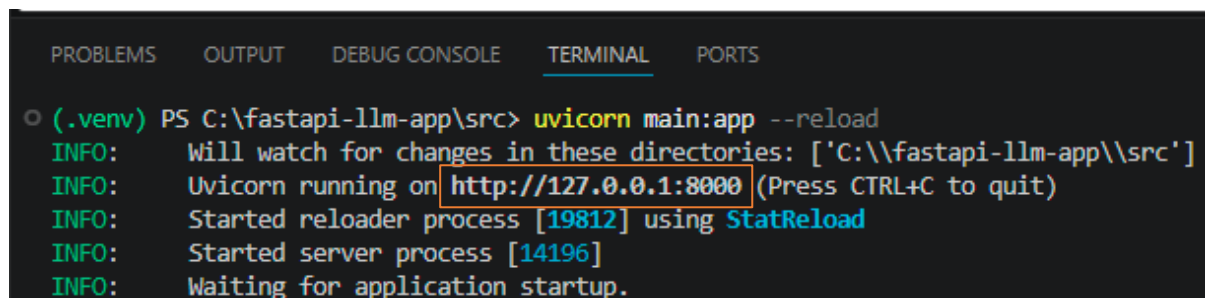
What to Observe

The server is now running locally at:

<http://127.0.0.1:8000> Keep this terminal running. If the terminal is closed or stopped, the FastAPI application will stop running.

Step 4 - Test the Home Endpoint

Open the following URL in your browser:



```

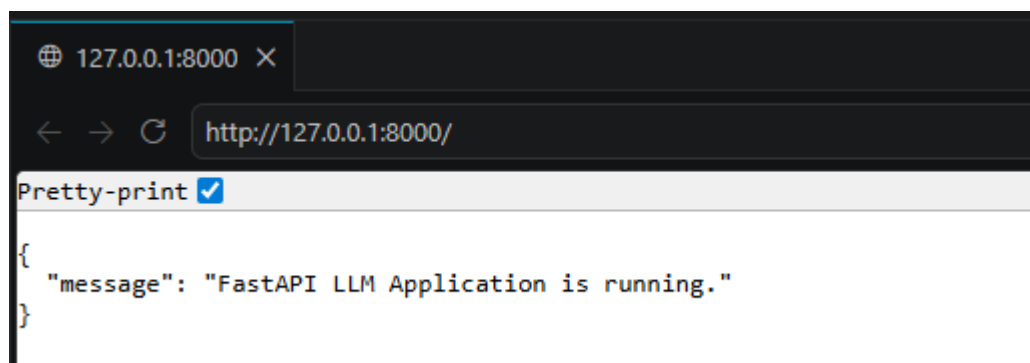
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(.venv) PS C:\fastapi-llm-app\src> uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\fastapi-llm-app\\src']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [19812] using StatReload
INFO:     Started server process [14196]
INFO:     Waiting for application startup.
  
```

Expected response:

```

{
  "message": "FastAPI LLM Application is running."
}
  
```

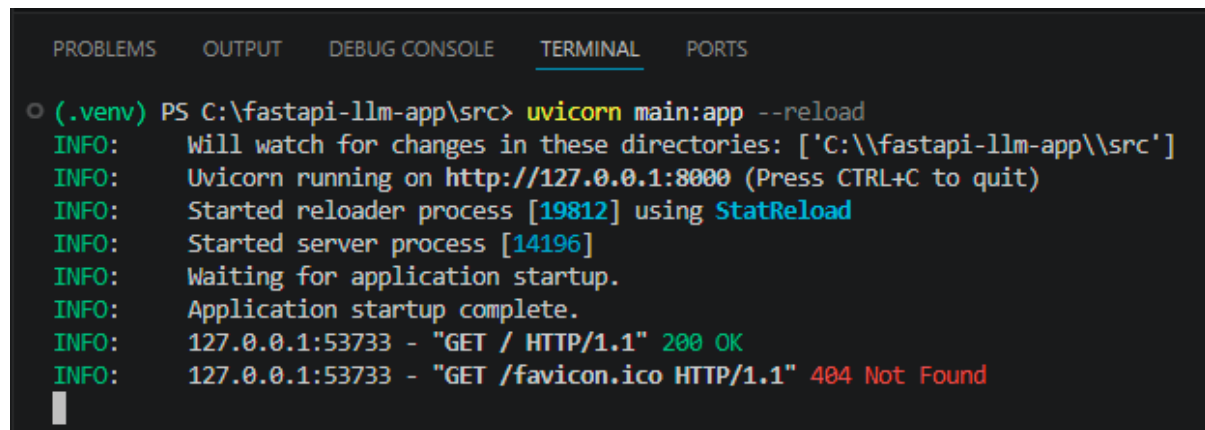


```

127.0.0.1:8000 X
http://127.0.0.1:8000/
Pretty-print [checked]
{
  "message": "FastAPI LLM Application is running."
}
  
```

This confirms that the FastAPI server is running successfully and the home endpoint is reachable.

Understanding the Server Logs



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) PS C:\fastapi-llm-app\src> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\fastapi-llm-app\\src']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [19812] using StatReload
INFO: Started server process [14196]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:53733 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:53733 - "GET /favicon.ico HTTP/1.1" 404 Not Found
```

GET / HTTP/1.1 200 OK

This indicates that the request was successfully processed and the application returned a valid response.

GET /favicon.ico HTTP/1.1 404 Not Found

Modern browsers automatically request a file named favicon.ico, which is used as the website icon displayed in browser tabs.

Since the application does not currently provide a favicon, FastAPI returns a 404 Not Found response. This is expected behavior and does not affect the functionality of the API.

Testing the API with Swagger UI

Step 1 – Open Swagger UI

<http://127.0.0.1:8000/docs>

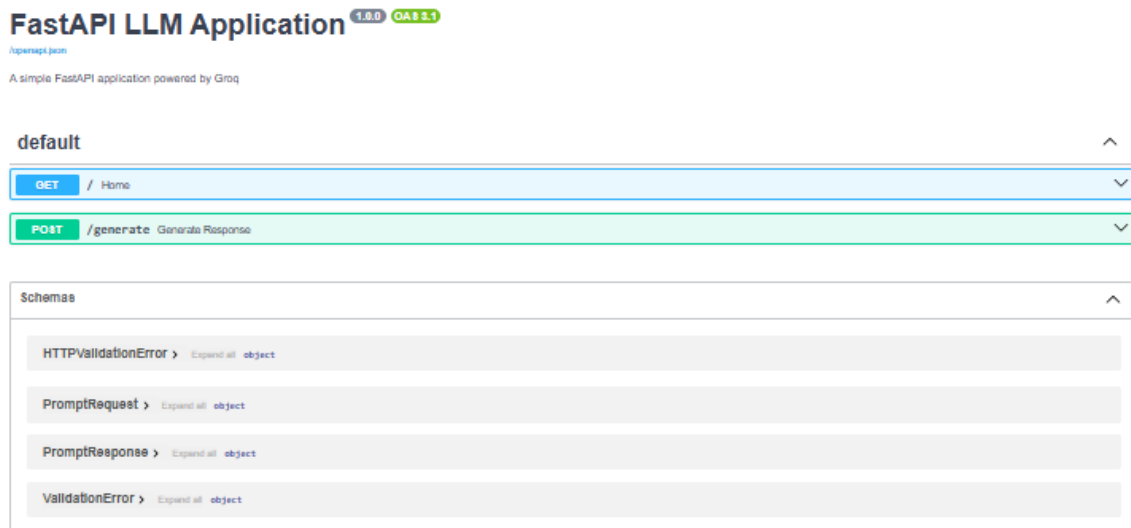
Swagger UI will appear automatically.

You should see:

- GET /
- POST /generate

Under the **Schemas** section, you should also see:

- PromptRequest
- PromptResponse
- HTTPValidationError
- ValidationError



Why Swagger UI Is Useful?

Swagger UI allows you to:

- Explore endpoints
- View request schemas
- Send test requests
- Inspect responses

without writing any client code.

Understanding the Generated Schemas

Because the API uses Pydantic models, FastAPI automatically generates documentation for both incoming requests and outgoing responses.

PromptRequest

PromptRequest represents the structure of the request body sent to the /generate endpoint.

Expected request:

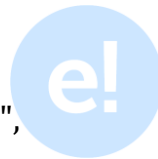
```
{
  "prompt": "Explain Agentic AI"
}
```

PromptResponse

PromptResponse represents the structure of the successful response returned by the API.

Expected response:

```
{
  "response": "Agentic AI refers to...",
  "model": "llama-3.3-70b-versatile",
  "status": "success"
}
```



Validation Schemas

FastAPI also generates validation schemas such as HTTPValidationError and ValidationError.

These are used when the request body does not match the expected schema

Note

If the application is started on a different host or port, replace **127.0.0.1:8000** with the appropriate address shown in the Uvicorn startup logs.

Example:

<http://localhost:9000/docs>

The **/docs** path remains the default Swagger UI endpoint provided by FastAPI.

Step 2 – Test the Generate Endpoint

Inside Swagger UI:

1. Expand POST /generate



2. Click Try it out



3. Enter:

```
{
  "prompt": "Explain Agentic AI in one paragraph."
}
```

POST
/generate
Generate Response

Parameters

No parameters

Request body required

Edit Value | Schema

```
{
  "prompt": "Explain Agentic AI in one paragraph."
}
```

4. Click Execute

POST
/generate
Generate Response

Parameters

No parameters

Request body required

Edit Value | Schema

```
{
  "prompt": "Explain Agentic AI in one paragraph."
}
```

Cancel

Reset

application/json

Execute

Expected Response:

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/generate' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "prompt": "Explain Agentic AI in one paragraph."
  }'
```

Request URL

```
http://127.0.0.1:8000/generate
```

Server response

Code Details

200

Response body

```
{
  "response": "Agentic AI refers to artificial intelligence systems that possess a degree of autonomy, allowing them to make decisions and take actions based on their own goals, motivations, and preferences. These systems are designed to be proactive, adaptive, and interactive, enabling them to learn from their environment, reason about their situation, and adjust their behavior accordingly. Agentic AI combines elements of machine learning, cognitive architectures, and decision-making frameworks to create intelligent agents that can operate with a high level of autonomy, making decisions that may not be explicitly programmed but are still aligned with their objectives. This approach has applications in areas such as robotics, autonomous vehicles, and smart homes, where AI systems need to be able to respond to dynamic situations and make decisions in real-time, often without human intervention.",
  "model": "llama-3.3-70b-versatile",
  "status": "success"
}
```

Download

Response body

```
{
  "response": "Agentic AI refers to artificial intelligence systems that possess a degree of autonomy, allowing them to make decisions and take actions based on their own goals, motivations, and preferences. These systems are designed to be proactive, adaptive, and interactive, enabling them to learn from their environment, reason about their situation, and adjust their behavior accordingly. Agentic AI combines elements of machine learning, cognitive architectures, and decision-making frameworks to create intelligent agents that can operate with a high level of autonomy, making decisions that may not be explicitly programmed but are still aligned with their objectives. This approach has applications in areas such as robotics, autonomous vehicles, and smart homes, where AI systems need to be able to respond to dynamic situations and make decisions in real-time, often without human intervention.",
  "model": "llama-3.3-70b-versatile",
  "status": "success"
}
```

Download

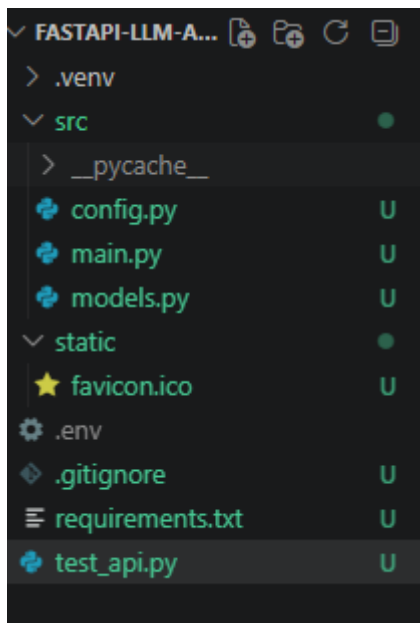
What to Observe

- The request is sent to the /generate endpoint.
- FastAPI validates the request body using the PromptRequest model.
- The prompt is sent to the Groq model.
- The generated output is returned in the response field.
- The model field shows which model generated the answer.
- The status field confirms that the request was completed successfully.

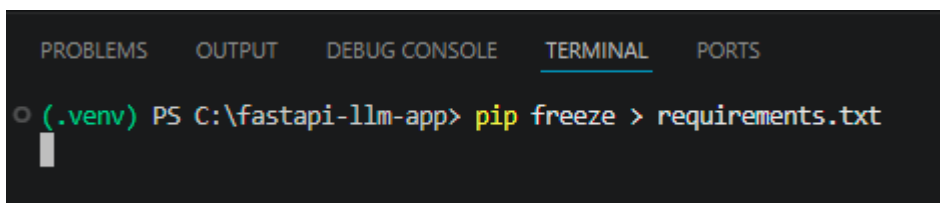
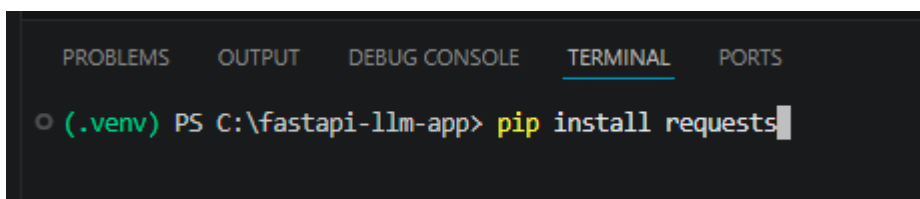
Step 3 - Test the API Using Python

In real-world AI applications, APIs are typically called from Python programs rather than directly from a terminal.

1. Create a new file **test_api.py**



2. Install the Requests Library and Update the dependency file:



What This Package Does?

The requests library allows Python applications to communicate with APIs using HTTP methods such as GET, POST, PUT, and DELETE.

It is one of the most commonly used libraries for consuming REST APIs in Python.

3. Add the following code in **test_api.py**:

```

test_api.py U X
test_api.py > ...
13     "Status Code:",
14     response.status_code
15 )
16
17 print(
18     "API Status:",
19     data.get("status")
20 )
21
22 print(
23     "Model Used:",
24     data.get("model")
25 )
26
27 print("Generated Response:")
28 print(
29     data.get("response")
30 )

```

Run:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● (.venv) PS C:\fastapi-llm-app> python test_api.py
Status Code: 200
API Status: success
Model Used: llama-3.3-70b-versatile
Generated Response:
Agentic AI refers to artificial intelligence systems that possess a degree of autonomy, allowing them to make decisions and take actions based on their own goals, motivations, and preferences. These systems are designed to be proactive, adaptive, and interactive, enabling them to learn from their environment, reason about their circumstances, and adjust their behavior accordingly. Agentic AI combines elements of machine learning, cognitive architectures, and decision-making theories to create intelligent agents that can operate with a significant level of independence, making them suitable for applications such as robotics, smart homes, and autonomous vehicles. By endowing AI systems with agency, researchers aim to create more flexible, resilient, and human-like intelligent systems that can effectively interact with their surroundings and achieve their objectives, even in complex and dynamic environments.
○ (.venv) PS C:\fastapi-llm-app>

```

Understanding the Code

- **requests.post()** sends an HTTP POST request to the FastAPI endpoint.
- **The json = {}** Sends the request body in JSON format. The API receives this prompt, validates it using the PromptRequest model, sends it to the Groq model, and returns a structured response.

- **data = response.json()** Converts the JSON response from the API into a Python dictionary.
- **response.status_code** Displays the HTTP status code returned by the API. A value of 200 means the request was successful.
- **data.get("status")** Reads the API-level status returned by the FastAPI endpoint.
- **data.get("model")** Displays the model used to generate the response.
- **data.get("response")** Extracts the generated LLM response.

Why This Matters

Most Agentic AI systems interact with APIs programmatically.

Instead of manually entering prompts through Swagger UI, agents, workflows, and applications send requests using Python code and process the returned responses automatically. This is the same approach used by AI frameworks such as LangChain, LangGraph, CrewAI, and AutoGen when interacting with external services.

Step 4 - Test Request Validation

FastAPI uses Pydantic models to validate incoming requests automatically.

1. Open Swagger UI

<http://127.0.0.1:8000/docs>

FastAPI LLM Application 1.0.0 QA 1.1

Appencept .json

A simple FastAPI application powered by Groq

default

GET / Home

POST /generate: Generate Response

Schemas

- HTTPValidationError > Expand all object
- PromptRequest > Expand all object
- PromptResponse > Expand all object
- ValidationError > Expand all object

2. Expand POST/ generate

POST /generate: Generate Response

Parameters

No parameters

Request body **required** application/json

Example Value | Schema

```
{
  "prompt": "string"
}
```

3. Click Try it out

POST /generate: Generate Response

Parameters

No parameters

Request body **required** application/json

Example Value | Schema

```
{
  "prompt": "string"
}
```

Try it out

4. Replace the request body:

With

Request body **required**

Edit Value | Schema

```
{ }
```

Expected Outcome

Code	Details
422	Error: Unprocessable Content Response body <pre>{ "detail": [{ "type": "missing", "loc": ["body", "prompt"], "msg": "Field required", "input": {} }] }</pre>

What Happened

The request was rejected because the required prompt field was missing from the request body.

The request never reached the endpoint function.

FastAPI automatically validated the request using the PromptRequest Pydantic model and returned a validation error before processing the request.

This built-in validation helps ensure that APIs receive properly structured input data.

Common Issues and Fixes

Issue	Cause	Fix
GROQ_API_KEY missing	.env not configured	Add API key
Model not found	Invalid model name	Use a valid Groq model
ModuleNotFoundError	Missing packages	Run pip install
127.0.0.1 not reachable	Server not running	Start Uvicorn
Validation error	Missing prompt field	Provide prompt in request

Conclusion

In this hands-on, you built and tested a structured FastAPI backend for an LLM-powered application. You created the project structure, configured environment variables, protected sensitive API keys, and defined both request and response schemas using Pydantic.

You then developed a `/generate` endpoint, integrated it with the Groq LLM API, and returned structured JSON responses containing the generated output, model name, and request status. You also tested the API through Swagger UI and Python code, while observing how FastAPI handles request validation, server logs, and API responses.

These skills form the foundation for building reliable AI backend services that can support chatbots, AI assistants, automation workflows, RAG pipelines, and production-ready Agentic AI systems.

